Chapter 02: Functions and Files

ENG3091

Khiem Nguyen

Contents

1 Why functions? 2
- 1.1 Copy, paste, repeat . . . 2
- 1.2 Don't Repeat Yourself (DRY) . . . 3

2 Defining and calling functions 3
- 2.1 Anatomy of a function . . . 3
- 2.2 Calling a function . . . 4
- 2.3 Functions can call functions (that call functions…) . . . 4
- 2.4 No functions inside functions . . . 5

3 Return values 5
- 3.1 Sending a value back to the caller . . . 5
- 3.2 Return values in action . . . 6
- 3.3 One call, one return value . . . 6
- 3.4 `main()` is special . . . 6
- 3.5 Forgetting to return a value . . . 7
- 3.6 `void` functions: nothing to return . . . 7
- 3.7 A `void` function cannot return a value . . . 8

4 Parameters and arguments 8
- 4.1 Parameters and arguments . . . 8
- 4.2 Zero, one or more parameters . . . 9
- 4.3 Pass by value . . . 9
- 4.4 Pass by value in action . . . 10
- 4.5 The swap that doesn't swap . . . 10
- 4.6 Coming from Python: what gets copied? . . . 11
- 4.7 Arguments can be any expression . . . 12
- 4.8 Watch out: argument order is not guaranteed . . . 13

5 Local variables: scope and lifetime 13
- 5.1 Local variables . . . 13
- 5.2 Lifetime: created, used, destroyed . . . 14
- 5.3 Scope: two variables, one name . . . 14
- 5.4 Scope: blocks inside functions . . . 15
- 5.5 Scope is not lifetime . . . 15
- 5.6 Scope is not lifetime: the proof . . . 16

6 Declarations and definitions 17
- 6.1 An innocent-looking program . . . 17
- 6.2 Fix 1: define before use . . . 17
- 6.3 Fix 2: a forward declaration . . . 18

6.4 Declarations and definitions . . . 18
6.5 Compiler happy, linker not . . . 18
6.6 Why bother with forward declarations? . . . 19

7 Programs with multiple files **19**
7.1 Why split a program into files? . . . 19
7.2 How g++ builds a multi-file program . . . 19
7.3 The same problem, now across two files . . . 20
7.4 The fix: forward-declare `add` in `main.cpp` . . . 20
7.5 Why are files compiled in isolation? . . . 21

8 Header files **21**
8.1 Forward declarations don't scale . . . 21
8.2 Header files . . . 21
8.3 Our first header . . . 21
8.4 Using the header . . . 22
8.5 Building step by step . . . 22
8.6 Header guards . . . 23
8.7 Header files: rules of thumb . . . 23

9 Wrapping up **23**
9.1 Summary . . . 23
9.2 Keep learning . . . 23

1 Why functions?

1.1 Copy, paste, repeat

Here is a program that reads two integers and adds them.

Listing 1 `repeat_yourself.cpp`

```cpp
#include <iostream>

int main() {
    int x{};
    std::cout << "Enter an integer: ";
    std::cin >> x;

    int y{};
    std::cout << "Enter an integer: ";
    std::cin >> y;

    std::cout << "Sum: " << x + y << '\n';
    return 0;
}
```

Lines 4–6 and 8–10 do exactly the same job. Now imagine reading *ten* numbers. Or changing the prompt text in every copy, and forgetting one.

1.2 Don't Repeat Yourself (DRY)

The golden rule: **write it once, give it a name, use it as often as you like.**

Listing 2 `dont_repeat_yourself.cpp`

```cpp
#include <iostream>

int get_value_from_user() {
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;
    return input;
}

int main() {
    int x{get_value_from_user()}; // first call
    int y{get_value_from_user()}; // second call

    std::cout << "Sum: " << x + y << '\n';
    return 0;
}
```

```
$ ./dont_repeat_yourself
Enter an integer: 3
Enter an integer: 4
Sum: 7
```

- `get_value_from_user()` is a **function**: a named, reusable block of code.
- `main()` **calls** it twice. Change the prompt once, and both calls change.

The rest of this chapter explains every piece of this program.

2 Defining and calling functions

2.1 Anatomy of a function

The general form

```cpp
return_type name(parameters) {
    statements
}
```

A real one

```cpp
int add(int x, int y) {
    return x + y;
}
```

- The **header** (first line) gives the function's name, what goes in (the **parameters**) and what comes out (the **return type**).
- The **body**, between `{` and `}`, holds the statements that run every time the function is called.
- `main()` is a function too. It's special only because every program starts there.

2.2 Calling a function

Listing 3 `call_function.cpp`

```cpp
#include <iostream>

void say_hello() { // definition of say_hello()
    std::cout << "  In say_hello(): Hello there!\n";
}

int main() {
    std::cout << "Starting main()\n";
    say_hello(); // first call
    say_hello(); // second call
    std::cout << "Ending main()\n";
    return 0;
}
```

```
Starting main()
  In say_hello(): Hello there!
  In say_hello(): Hello there!
Ending main()
```

- A **function call** is the name followed by parentheses: `say_hello();`
- `main()` pauses, `say_hello()` runs, then `main()` carries on right after the call.

`main()` is the **caller** and `say_hello()` is the **callee**. (`void` means "gives nothing back". More on that soon.)

2.3 Functions can call functions (that call functions...)

Listing 4 `function_call_function.cpp`

```cpp
#include <iostream>

void do_b() { std::cout << "    In do_b()\n"; }

void do_a() {
    std::cout << "  Starting do_a()\n";
    do_b();
    std::cout << "  Ending do_a()\n";
}

int main() {
    std::cout << "Starting main()\n";
    do_a();
    std::cout << "Ending main()\n";
    return 0;
}
```

```
Starting main()
  Starting do_a()
    In do_b()
  Ending do_a()
Ending main()
```

Read the output from top to bottom: every call returns to exactly the place it was called from.

2.4 No functions inside functions

Listing 5 `nested_function.cpp`

```cpp
#include <iostream>

int main() {
    // Illegal: foo() is defined inside main()
    void foo() { std::cout << "foo!\n"; }

    foo();
    return 0;
}
```

```
error: a function-definition is not allowed here before '{' token
```

A function defined inside another function is a **nested function**. C++ doesn't allow them: every function definition lives at the top level of a file. (`foo` is the name programmers use when the name doesn't matter.)

> **i** Coming from Python
>
> Python and MATLAB let you define a function inside another one. C++ does not.

3 Return values

3.1 Sending a value back to the caller

A function can hand **one value** back to whoever called it. It needs two things:

1. a **return type** in the header, saying what kind of value comes back (e.g. `int`);
2. a **return statement** in the body, saying which value: `return input;`

When the return statement runs:

1. The expression after `return` is evaluated.
2. A **copy** of the result is sent back to the caller. This copy is the **return value**.
3. The function ends, and the caller carries on.

3.2 Return values in action

Listing 6 `return_value.cpp`

```cpp
#include <iostream>

int get_value_from_user() { // returns an int
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;
    return input; // send a copy of input back
}

int main() {
    // num is initialised with the return value
    int num{get_value_from_user()};

    std::cout << num << " x 2 = " << num * 2 << '\n';
    return 0;
}
```

```
$ ./return_value
Enter an integer: 21
21 x 2 = 42
```

On line 12, the call `get_value_from_user()` produces the value it returns (here `21`), and that value initialises `num`.

3.3 One call, one return value

- Each call returns exactly **one** value.
- Need two results from one function? There are ways around it (references, structs and classes), and we'll meet them in later chapters.

> **i** Coming from Python
>
> `return total, average` in Python *looks* like two values, but it actually returns **one** tuple. C++ can play the same trick by returning one object that holds several values, such as a struct or a `std::pair`. Later!

3.4 `main()` is special

- `main()` must return an `int`. The value goes back to the operating system: `0` means "everything went fine".
- `main()` is the only value-returning function allowed to skip `return`: reaching its closing `}` returns `0`. We'll keep writing `return 0;` anyway. It's clearer.
- Never call `main()` yourself. The C++ standard forbids it, although g++ only complains if you compile with `-pedantic-errors`.

```cpp
void main() { // error: '::main' must return 'int'
}
```

3.5 Forgetting to return a value

Listing 7 `missing_return.cpp`

```cpp
#include <iostream>

int get_answer() {
    std::cout << "Thinking...\n";
    // Oops: we forgot to write "return 42;"
}

int main() {
    std::cout << get_answer() << '\n';
    return 0;
}
```

```
$ g++ -std=c++20 -Wall missing_return.cpp
warning: no return statement in function
returning non-void [-Wreturn-type]
$ ./a.out
Thinking...
Illegal instruction
```

- This **compiles**: g++ only gives a *warning*. But the program has **undefined behaviour**, which means anything can happen. On my machine, it crashed.
- Warnings are free bug reports. Turn them on and read them!

> * Best practice
>
> Compile with `g++ -std=c++20 -Wall -Wextra` and treat every warning as a bug to fix.

3.6 `void` functions: nothing to return

Listing 8 `void_return.cpp`

```cpp
#include <iostream>

void say_hi() { // void: returns no value
    std::cout << "Hi\n";
    std::cout << "I'm Khiem\n";
    return; // go back to the caller now

    std::cout << "This line never runs\n";
}

int main() {
    say_hi();
    std::cout << "Back in main()\n";
    return 0;
}
```

```
Hi
I'm Khiem
Back in main()
```

- `void` means "this function returns no value".
- A `void` function needs no `return`: it goes back to the caller at its closing `}`.
- `return;` goes back *immediately*, so line 8 never runs.

On its own, an early `return;` is pointless. Together with `if` statements (Chapter 03) it becomes really useful.

3.7 A `void` function cannot return a value

No surprise here: if you promise to return nothing, the compiler holds you to your promise.

```
1 void say_hi() {
2     std::cout << "Hi\n";
3     return 5; // we promised void!
4 }
```

```
error: return-statement with a value, in function returning 'void'
```

4 Parameters and arguments

4.1 Parameters and arguments

- A **parameter** is a variable declared in the function header. It's a placeholder for a value the caller will supply.
- An **argument** is the actual value the caller passes in the function call.

  Memory trick: **P**arameters are **P**laceholders; **A**rguments are **A**ctual values.

```
1 void print_double(int value) { // value is a PARAMETER
2     std::cout << value * 2 << '\n';
3 }
4
5 int main() {
6     print_double(21); // 21 is an ARGUMENT
7     return 0;
8 }
```

4.2 Zero, one or more parameters

Listing 9 `function_parameters.cpp`

```cpp
#include <iostream>

// No parameters: needs nothing from the caller
void say_hello() { std::cout << "Hello World\n"; }

// One parameter: the caller supplies x
void print_value(double x) { std::cout << x << '\n'; }

// Two parameters: the caller supplies x and y
double add(double x, double y) { return x + y; }

int main() {
    say_hello();                        // no arguments
    print_value(6.5);                   // one argument
    std::cout << add(2.0, 3.5) << '\n'; // two arguments
    return 0;
}
```

```
Hello World
6.5
5.5
```

A call must supply one argument for each parameter, in the same order.

4.3 Pass by value

When a function is called:

1. each parameter is created as a brand-new local variable, and
2. it is initialised with a **copy** of the matching argument.

This is called **pass by value**, and parameters like these are **value parameters**.

> **!** Key idea
>
> With pass by value, the function works on its **own copy** of each argument. Whatever it does to the copy stays inside the function.

4.4 Pass by value in action

Listing 10 `double_a_number.cpp`

```cpp
#include <iostream>

int get_value_from_user() {
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;
    return input;
}

void print_double(int value) { // one int parameter
    std::cout << value << " x 2 = " << value * 2 << '\n';
}

int main() {
    int num{get_value_from_user()};
    print_double(num);

    print_double(get_value_from_user());
    return 0;
}
```

```
$ ./double_a_number
Enter an integer: 7
7 x 2 = 14
Enter an integer: 21
21 x 2 = 42
```

On line 18, the return value of one call becomes the argument of another.

4.5 The swap that doesn't swap

What does this program print? Think before you look!

Listing 11 `pass_by_value.cpp`

```cpp
#include <iostream>

void try_to_swap(int x, int y) {
    int temp{x};
    x = y;
    y = temp;
    std::cout << "In try_to_swap(): " << x << ' ' << y << '\n';
}

int main() {
    int x{100};
    int y{200};
    try_to_swap(x, y);
    std::cout << "Back in main():   " << x << ' ' << y << '\n';
    return 0;
}
```

```
In try_to_swap(): 200 100
Back in main():   100 200
```

`try_to_swap()` swapped its own **copies**; `main()`'s `x` and `y` never changed. To change the caller's variables we need *references* (Chapter 07).

4.6 Coming from Python: what gets copied?

Listing 12 `pass_by_value.py`

```python
def add_ten(numbers):
    numbers[0] += 10


my_list = [5]
add_ten(my_list)
print(my_list)  # [15]
```

- In Python, the parameter and the argument refer to the **same object**. Change the list inside the function and the caller sees the change.
- In C++, pass by value makes a **separate copy** of the whole argument, even if it's a big container of numbers. Nothing the function does to its copy can reach the caller.

4.7 Arguments can be any expression

Listing 13 `pass_by_value_more.cpp`

```cpp
#include <iostream>

int add(int x, int y) { return x + y; }

int multiply(int x, int y) { return x * y; }

int main() {
    std::cout << add(4, 5) << '\n';          // x = 4, y = 5
    std::cout << add(1 + 2, 3 * 4) << '\n'; // x = 3, y = 12

    int a{5};
    std::cout << add(a, a) << '\n'; // x = 5, y = 5

    std::cout << add(1, multiply(2, 3)) << '\n'; // 1 + (2 * 3)
    std::cout << add(1, add(2, 3)) << '\n';      // 1 + (2 + 3)
    return 0;
}
```

```
9
15
10
7
6
```

Each argument is evaluated first, and the resulting value is copied into the parameter. An argument can even be another function call, like `multiply(2, 3)`.

4.8 Watch out: argument order is not guaranteed

Listing 14 `evaluation_order.cpp`

```cpp
#include <iostream>

int get_value_from_user() {
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;
    return input;
}

void print_diff(int x, int y) {
    std::cout << x << " - " << y << " = " << x - y << '\n';
}

int main() {
    print_diff(get_value_from_user(), get_value_from_user());
    return 0;
}
```

```
$ ./evaluation_order
Enter an integer: 10
Enter an integer: 3
3 - 10 = -7
```

Wait, what? We typed `10` first!

C++ does **not** fix the order in which arguments are evaluated. g++ called the *second* `get_value_from_user()` first.

The fix: read the values into variables in separate statements, then pass the variables.

5 Local variables: scope and lifetime

5.1 Local variables

Variables defined inside a function body are **local variables**. Parameters count as local variables too.

```cpp
int add(int x, int y) { // parameters x and y are local variables
    int z{x + y};       // z is a local variable
    return z;
}
```

Every variable raises two different questions:

	Scope	**Lifetime**
The question	*Where* in the code can I use the name?	*When* does the variable exist in memory?
Decided at	compile time	run time
A local variable's	from its definition to the end of its block	from its creation to the end of its block

5.2 Lifetime: created, used, destroyed

Listing 15 `local_variable_lifetime.cpp`

```cpp
#include <iostream>

void print_double(int value) { // value is created here
    int result{value * 2};     // result is created here
    std::cout << result << '\n';
} // result, then value, are destroyed here

int main() {
    int number{21}; // number is created here
    print_double(number);
    return 0;
} // number is destroyed here
```

- Parameters are created when the function starts; other local variables when their definition runs.
- At the closing `}`, local variables are destroyed in the **reverse** order of creation.
- "Destroyed" usually just means the memory can be reused. For class objects, a special function called a **destructor** runs first (Chapter 12).

5.3 Scope: two variables, one name

Listing 16 `local_scope_01.cpp`

```cpp
#include <iostream>

void print_x() {
    int x{42}; // this x belongs to print_x()
    std::cout << "In print_x(): x = " << x << '\n';
}

int main() {
    int x{7}; // a different x, belonging to main()
    print_x();
    std::cout << "In main():    x = " << x << '\n';
    return 0;
}
```

```
In print_x(): x = 42
In main():    x = 7
```

- Each function has its own `x`. They just happen to share a name.
- A local name only means something inside its own function, so you can reuse good names freely.

5.4 Scope: blocks inside functions

Listing 17 `local_scope_02.cpp`

```cpp
#include <iostream>

int main() {
    int a{5};

    { // a nested block starts here
        int b{10};
        std::cout << a + b << '\n'; // a and b in scope
    } // b goes out of scope (and is destroyed) here

    std::cout << b << '\n'; // error: b is not in scope
    return 0;
}
```

```
error: 'b' was not declared in this scope
```

A name can be used from its definition to the end of the innermost `{ }` around it, hence the name **block scope**. Break that rule and you get a **compile-time** error.

> **i** Coming from Python
>
> Python has no block scope: a variable created inside an `if` or `for` body is still usable after it, until the function ends. In C++, it's gone at the `}`.

5.5 Scope is not lifetime

Listing 18 `scope_vs_lifetime.cpp`

```cpp
#include <iostream>

void second_function() {
    // x is alive, but OUT OF SCOPE here:
    // the name x cannot be used in this function.
}

void first_function() {
    int x{10}; // x's scope and lifetime start here
    second_function();
    std::cout << x << '\n'; // x is in scope again
} // x's scope and lifetime end here

int main() {
    first_function();
    return 0;
}
```

- While `second_function()` runs, `x` is **out of scope**: the name `x` can't be used there.

- But `x` is still **alive**: `first_function()` hasn't finished yet, and line 11 still prints `10`.

I don't think you believe me. Let me prove it to you!

5.6 Scope is not lifetime: the proof

Listing 19 `scope_vs_lifetime_proof.cpp`

```cpp
#include <iostream>

void second_function(int* address_of_x) {
    // The NAME x is out of scope, but we know its ADDRESS
    std::cout << "x is at " << address_of_x << '\n';
    std::cout << "x holds " << *address_of_x << '\n';
}

void first_function() {
    int x{10};           // x is alive from here...
    second_function(&x); // &x means "the address of x"
} // ...to here

int main() {
    first_function();
    return 0;
}
```

```
x is at 0x7fffdc7401c4
x holds 10
```

No *name* `x` in sight, yet `10` is still there. So `x` is alive!

> **i** Sneak peek
>
> `&x` is the address of `x`; `*address_of_x` reads the value stored there. These are **pointers** (Chapter 08).

Your address will differ. Prefer a debugger? Pause inside `second_function()` and you'll see `x` alive and well in `first_function()`.

6 Declarations and definitions

6.1 An innocent-looking program

Listing 20 `without_forward_declaration.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "3 + 4 = " << add(3, 4) << '\n';
    return 0;
}

int add(int x, int y) { return x + y; }
```

```
error: 'add' was not declared in this scope
```

- The compiler reads a file **from top to bottom**.
- When it reaches `add(3, 4)` on line 4, it has never heard of `add`. The definition further down comes too late.

6.2 Fix 1: define before use

Listing 21 `reorder_function_defs.cpp`

```cpp
#include <iostream>

int add(int x, int y) { // defined first...
    return x + y;
}

int main() {
    std::cout << "3 + 4 = " << add(3, 4) << '\n'; // ...used after
    return 0;
}
```

- Simple! But with many functions calling each other, you have to work out a valid order by hand.
- Sometimes no valid order exists: think of two functions that call each other.

6.3 Fix 2: a forward declaration

Listing 22 `with_forward_declaration.cpp`

```cpp
#include <iostream>

int add(int x, int y); // forward declaration (note the ;)

int main() {
    std::cout << "3 + 4 = " << add(3, 4) << '\n'; // OK now
    return 0;
}

int add(int x, int y) { // the definition can come later
    return x + y;
}
```

- `int add(int x, int y);` is a **forward declaration**, also called a **function prototype**: the function header followed by a semicolon.
- It tells the compiler "`add` exists, and this is how to call it. The body comes later."
- The parameter names are optional in a declaration (`int add(int, int);`), but good names document what each parameter is for.

6.4 Declarations and definitions

- A **declaration** introduces a name and its type: `int add(int x, int y);`
- A **definition** also provides the body: `int add(int x, int y) { return x + y; }`
- Every definition is also a declaration.

  Declare as often as you like; define **exactly once**. That's the gist of C++'s *one definition rule*.

What if we declare a function but never define it?

Listing 23 `declared_not_defined.cpp`

```cpp
#include <iostream>

int add(int x, int y); // declared...

int main() {
    std::cout << "3 + 4 = " << add(3, 4) << '\n';
    return 0;
}

// ...but never defined anywhere!
```

6.5 Compiler happy, linker not

```
$ g++ -std=c++20 declared_not_defined.cpp
/usr/bin/ld: in function `main':
```

```
undefined reference to `add(int, int)'
collect2: error: ld returned 1 exit status
```

- The **compiler** is happy: it trusted the declaration.
- Then the **linker** (`ld`) tries to connect the call `add(3, 4)` to the body of `add`, and there is no body to be found.
- Learn to tell them apart: *"was not declared in this scope"* comes from the compiler; *"undefined reference"* comes from the linker.

6.6 Why bother with forward declarations?

- **The main reason:** the function is defined in **another file**. Reordering can't help when caller and callee live in different files. That's the next section!
- **Two functions that call each other:** no order works, but a declaration does.
- **Readability:** you can arrange functions in whatever order reads best, for example `main()` first.

> **i** Coming from Python
>
> Python looks up a name when a line *runs*. A function body can call a function defined further down the file, as long as that definition has run by the time the call happens. C++ checks every name when it *compiles*, from top to bottom.

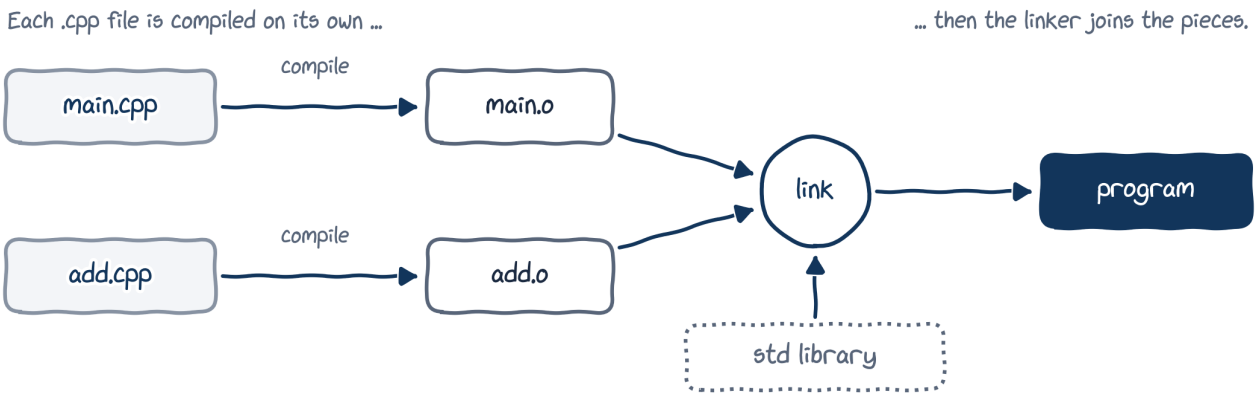
7 Programs with multiple files

7.1 Why split a program into files?

- **Organisation:** related functions live together, e.g. `geometry.cpp`, `input.cpp`.
- **Reuse:** the same file can serve several programs.
- **Teamwork:** different people can work on different files.
- **Speed:** after an edit, only the changed file needs recompiling.

Adding a new file to your project in the IDE is a couple of clicks. That's much easier to show than to put on a slide, so watch me do it live (it's also in the recorded video).

7.2 How g++ builds a multi-file program



- **Compile:** each `.cpp` file is compiled *on its own* into an **object file** (`.o`). While compiling one file, the compiler knows nothing about the others.
- **Link:** the **linker** joins the object files (plus the standard library) into one program, connecting each function call to its body.

7.3 The same problem, now across two files

Listing 24 `add.cpp`

```cpp
int add(int x, int y) { return x + y; }
```

Listing 25 `main.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "3 + 4 = " << add(3, 4) << '\n';
    return 0;
}
```

```
$ g++ -std=c++20 main.cpp add.cpp -o program
main.cpp:4:32: error: 'add' was not declared in this scope
```

- `main.cpp` is compiled on its own, so the compiler never sees `add.cpp` while compiling it.
- Reordering is no help: `add` lives in a different file.

7.4 The fix: forward-declare `add` in `main.cpp`

Listing 26 `add.cpp`

```cpp
int add(int x, int y) { return x + y; }
```

Listing 27 `main.cpp`

```cpp
#include <iostream>

int add(int x, int y); // add() is defined in another file

int main() {
    std::cout << "3 + 4 = " << add(3, 4) << '\n';
    return 0;
}
```

```
$ g++ -std=c++20 -Wall main.cpp add.cpp -o program
$ ./program
3 + 4 = 7
```

List **every** `.cpp` file on the command line. (On Windows, the program is `program.exe`.)

7.5 Why are files compiled in isolation?

That short memory is deliberate:

- Files can be compiled in any order, or several at once.
- After an edit, only the changed file needs recompiling. In a big project that saves minutes, sometimes hours.
- A name used privately in one file can't clash with the same name in another.

8 Header files

8.1 Forward declarations don't scale

- 5 files, each needing 10 forward declarations: 50 declarations to copy, paste and keep in sync.
- 100 files needing 100 each: 10,000. Change one function's parameters and enjoy fixing them all.

The C++ answer: write the declarations **once**, in a **header file**, and `#include` it wherever it's needed.

8.2 Header files

- A header file (`.h`, sometimes `.hpp`) holds **declarations**, not function bodies.
- `#include "add.h"` asks the **preprocessor** to paste the text of `add.h` in place of that line, before the compiler sees the file.
- You've used headers since your first program: `#include <iostream>` pastes in the declarations of `std::cout` and friends.

> ! Key idea
>
> Header files let you write each declaration once and reuse it everywhere.

8.3 Our first header

Listing 28 `add.h`

```
#ifndef ADD_H
#define ADD_H

int add(int x, int y);

#endif
```

Listing 29 `add.cpp`

```
// Including our own header lets the compiler
// check that declaration and definition match.
#include "add.h"

int add(int x, int y) { return x + y; }
```

- `add.h` holds the declaration of `add()`. The `#ifndef`/`#define`/`#endif` lines are a **header guard**, explained in two slides.

- `add.cpp` includes its own header, so the compiler checks that the declaration and the definition match.

8.4 Using the header

Listing 30 `main.cpp`

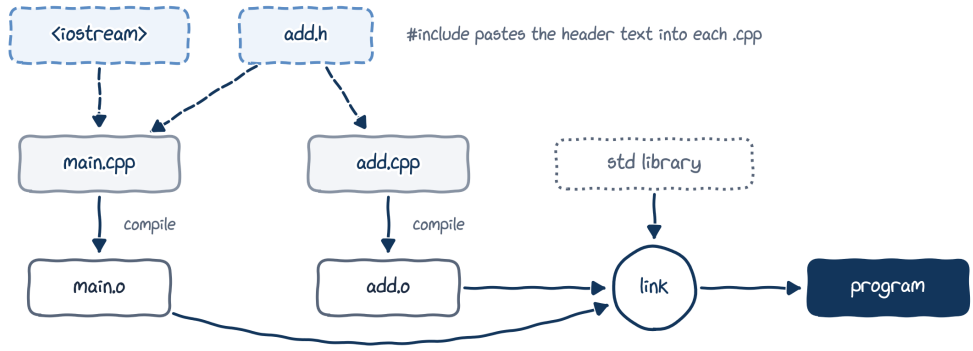
```cpp
#include "add.h" // pastes in the declaration of add()
#include <iostream>

int main() {
    std::cout << "3 + 4 = " << add(3, 4) << '\n';
    return 0;
}
```

```
$ g++ -std=c++20 -Wall main.cpp add.cpp -o program
$ ./program
3 + 4 = 7
```

- `"add.h"` in quotes: a header from **your project**, searched for next to your file first.
- `<iostream>` in angle brackets: a header from the **standard library**.
- Headers are never compiled on their own, so they never go on the command line.

8.5 Building step by step




```
$ g++ -std=c++20 -c main.cpp     # compile only: makes main.o
$ g++ -std=c++20 -c add.cpp      # compile only: makes add.o
$ g++ main.o add.o -o program    # link: makes program
```

`-c` means “compile, but don’t link”. Change `add.cpp` and only `add.o` needs rebuilding. Build tools like `make` and CMake automate exactly this.

8.6 Header guards

Listing 31 `add.h`

```cpp
#ifndef ADD_H  // if ADD_H isn't defined yet...
#define ADD_H  // ...define it and keep everything up to #endif

int add(int x, int y);

#endif         // second time round, all of this is skipped
```

- A header can end up included twice in the same `.cpp` file, often indirectly through another header. Its text would then be pasted twice.
- Repeating a *declaration* is harmless, but soon our headers will hold *class definitions* (Chapter 11), and defining something twice is an error.
- With a guard, the second `#include` pastes nothing. Give every header a guard with a unique name (`ADD_H`, `GEOMETRY_H`, …).
- You'll also see `#pragma once`: one line, same job, supported by all major compilers, though not part of the C++ standard.

8.7 Header files: rules of thumb

- Pair them up: `add.cpp` shares its functions through `add.h`.
- Declarations go in the header; definitions go in the `.cpp` file. (Classes and templates bend this rule; we'll see why later.)
- Every header gets a header guard.
- `#include` header files only. Never `#include` a `.cpp` file.
- Include what you use: if a file uses `std::cout`, it includes `<iostream>` itself rather than relying on another header to do it.

9 Wrapping up

9.1 Summary

- Functions let you write code once and use it many times (DRY).
- A function has a **header** (return type, name, parameters) and a **body**.
- Arguments are **copied** into parameters: this is *pass by value*.
- `return` sends one value back to the caller; `void` functions send none.
- **Scope** is *where* a name can be used (compile time); **lifetime** is *when* a variable exists (run time).
- The compiler reads each file top to bottom and one file at a time: declare before you use.
- Header files share declarations between files; header guards make them safe to include.

9.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Naming collisions and an introduction to namespaces
2. Introduction to the preprocessor
3. Header guards, in more depth

Next up, Chapter 03: control flow. With `if`, `switch` and loops, our functions can finally make decisions.